

Reusable Multi-Language Product Modifier Library

Database tables, relationships, resolution rules, cart validation, and checkout snapshot flow

Architecture summary

The schema contains 19 Store-scoped tables split into three layers: 10 reusable library tables, 7 product configuration tables, and 2 transactional tables. A modifier is defined once and reused across products; product assignments store only restrictions and overrides.

Lifecycle at a glance



Identity and tenancy

Table	Purpose	Key behavior
store_id	Present throughout the schema for tenant isolation.	Translations and junction tables retain Store-aware composite relationships.
Internal IDs	Used for relational joins and database consistency.	Two product/value junctions also retain internal modifier_id for composite integrity.
Public ULIDs	Exposed only for externally addressable entities.	Category, modifier definition, modifier value, product group, and product assignment.
locale	Uses the existing 35-character locale convention.	Translation records support lock_it behavior consistent with the project.

1. Reusable modifier library

This layer owns canonical modifier content. Product records reference it instead of copying names, values, validation rules, or default prices.

Table	Purpose	Key behavior
<code>modifier_library_categories</code>	Optional categories for organizing reusable modifiers.	One category can contain many definitions; public ULID.
<code>modifier_library_category_translations</code>	Localized category names and descriptions.	Unique per Store, category, and locale.
<code>modifier_definitions</code>	Reusable master definition with code, input type, configuration, and status.	Defined once and assigned to many products; public ULID.
<code>modifier_translations</code>	Localized modifier name, description, and help text.	Library translation fallback source.
<code>modifier_values</code>	Selectable values such as sizes, colors, or add-ons.	Belongs to one definition; public ULID.
<code>modifier_value_translations</code>	Localized value labels and descriptions.	Falls back to the Store default locale, then value code.
<code>modifier_validation_rules</code>	Reusable input constraints: length, pattern, numeric range, or file restrictions.	Evaluated when cart selections are submitted.
<code>modifier_validation_rule_translations</code>	Localized validation messages.	Keeps storefront errors translatable.
<code>modifier_price_adjustments</code>	Library-level pricing for selecting a modifier.	Filtered by currency, audience, activity dates, and status.
<code>modifier_value_price_adjustments</code>	Library-level pricing for an individual value.	Combined with the effective modifier-level component.

Reuse principle

A definition such as Engraving, Size, or Gift wrap is created once. The same definition and values can then be assigned to any number of products.

2. Product configuration

The product layer expresses placement and differences from the library. It can reorder values, restrict availability, override localized labels, and replace default prices.

Table	Purpose	Key behavior
product_modifier_groups	Groups modifiers on a product, such as Extras or Choose a size.	Supports ordering and public ULID.
product_modifier_group_translations	Localized product group names.	Unique per Store, group, and locale.
product_modifier_assignments	Connects a product to a reusable modifier.	Stores group, order, required state, min/max, settings, and public ULID.
product_modifier_assignment_translations	Product-specific localized label and description overrides.	Takes precedence over library text in the requested locale.
product_modifier_value_assignments	Enables, disables, or reorders values for a product.	Restricts the reusable value set without duplicating it.
product_modifier_price_overrides	Product-specific modifier-level price replacement.	Replaces the matching library component; it does not add a second component.
product_modifier_value_price_overrides	Product-specific value-level price replacement.	Replaces the matching library value component.

3. Cart and order records

Table	Purpose	Key behavior
cart_item_modifier_selections	Validated selections attached to a cart item.	Multi-select stores one row per selected value; free form uses JSON input.
order_item_modifier_snapshots	Immutable checkout copy of localized names, codes, ULIDs, input, and pricing.	Source foreign keys are nullable and use SET NULL; order history never cascades away.

Historical safety

Order snapshots remain readable when products, assignments, modifier definitions, or values are renamed or removed. Source references may become NULL, but captured codes, localized labels, input, prices, currency, and locale remain unchanged.

4. Relationship map

The relationships are intentionally layered: canonical library data flows into product assignments; resolved assignments create cart selections; checkout converts those selections into immutable order snapshots.

Relationship	Meaning
Category -> modifier definition	One optional category organizes many reusable modifier definitions.
Modifier definition -> values	One definition exposes zero or more selectable reusable values.
Modifier definition -> validation rules	Rules describe how free-form or media input must be validated.
Modifier/value -> library prices	Library price components provide the default pricing behavior.
Product + modifier -> assignment	The assignment is the reusable link and carries product-specific behavior.
Product -> group -> assignment	Groups are optional and control storefront organization and order.
Assignment -> value assignments	The product can enable, disable, and reorder the modifier's values.
Assignment -> product price overrides	Product pricing replaces the corresponding library component.
Assignment -> cart selections	Only server-validated effective selections are stored.
Cart selection -> order snapshot	Checkout copies display and pricing data exactly once.

Deletion behavior

Transactional history is protected from source data changes. Order snapshot source foreign keys are nullable and use SET NULL rather than cascading deletion.

5. Runtime resolution and validation

Storefront resolution

The resolved endpoint loads active product assignments, definitions, allowed values, requested-locale content, Store-default fallbacks, and matching price rows. It returns a storefront-ready DTO rather than exposing raw internal rows.

Resolution	Precedence or calculation
Modifier label	Requested-locale assignment override -> requested-locale library translation -> Store-default library translation -> modifier code.
Value label	Requested-locale library translation -> Store-default library translation -> value code.
Modifier price	Matching product modifier override replaces the matching library modifier component.
Value price	Matching product value override replaces the matching library value component.
Final adjustment	Effective modifier component + effective selected-value component. Percentage pricing uses product base price.
Audience match	Exact sales channel/customer group match wins over broader active pricing for the same currency and date.

Cart validation

1. Re-resolve the active product assignment; never trust stale client configuration.
2. Enforce required state and minimum/maximum selection counts.
3. Confirm every value belongs to the modifier and is enabled for the product.
4. Validate free-form input and same-Store ready media for file/image modifiers.
5. Ignore client-submitted prices and recalculate the effective adjustment server-side.
6. Store one cart row per selected value, or JSON input for free-form selections.

6. API and implementation reference

Primary resolved endpoint

Storefront read

GET /api/v1/store/products/{product}/modifiers/resolved
?locale=en¤cy=GBP

Returns assignment and modifier ULIDs, code, type, localized name, required/min/max settings, allowed values, and effective price adjustments.

Service sequence



Source files

Reference	Repository path
Library migration	Modules/Catalog/database/migrations/2026_08_25_001000_create_modifier_library_tables.php
Product integration migration	Modules/Catalog/database/migrations/2026_08_25_001100_create_product_modifier_assignment_tables.php
Cart/order migration	Modules/Catalog/database/migrations/2026_08_25_001200_create_cart_and_order_modifier_tables.php
Catalog reference	docs/catalog.md
API manual	docs/api-manual.md

Current integration boundary

Cart and order integration services and storage are prepared, but this repository does not currently contain Cart, Orders, Sales Channel, or Customer Group modules. Checkout HTTP routes therefore remain unwired, and audience IDs are not accepted through the public API until those modules expose ULIDs.